Midterm Examination, Autumn-2023

Course Title: Data Structures

Course Code: CSE-2321

(This PDF contains ChatGPT generated answers. While it aims to provide useful insights, some content may be inaccurate or imprecise. Use this as a study guide and confirm facts when needed.)

1.a) A company maintains an employee database with the following data for each employee:
Employee ID, Name, Department, Salary, Joining Date
i) State the **entities, attributes** and **entity set** of the list.
ii) Which attribute can serve as the **primary key** for the list?

Ans: i)

Entity: Employee

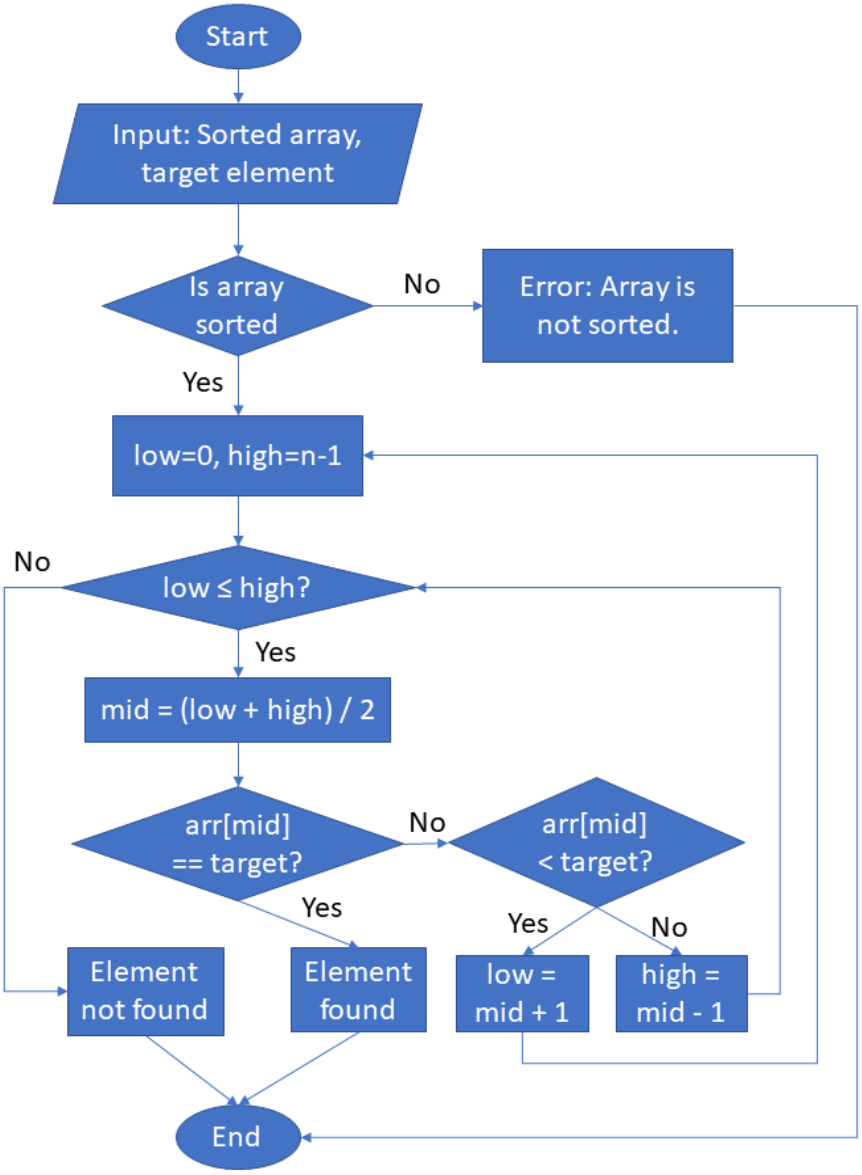
Attributes: Employee ID, Name, Department, Salary, Joining Date

Entity Set: Set of all employees in the database

ii) **Primary Key:** `Employee ID` (because it's unique for each employee)

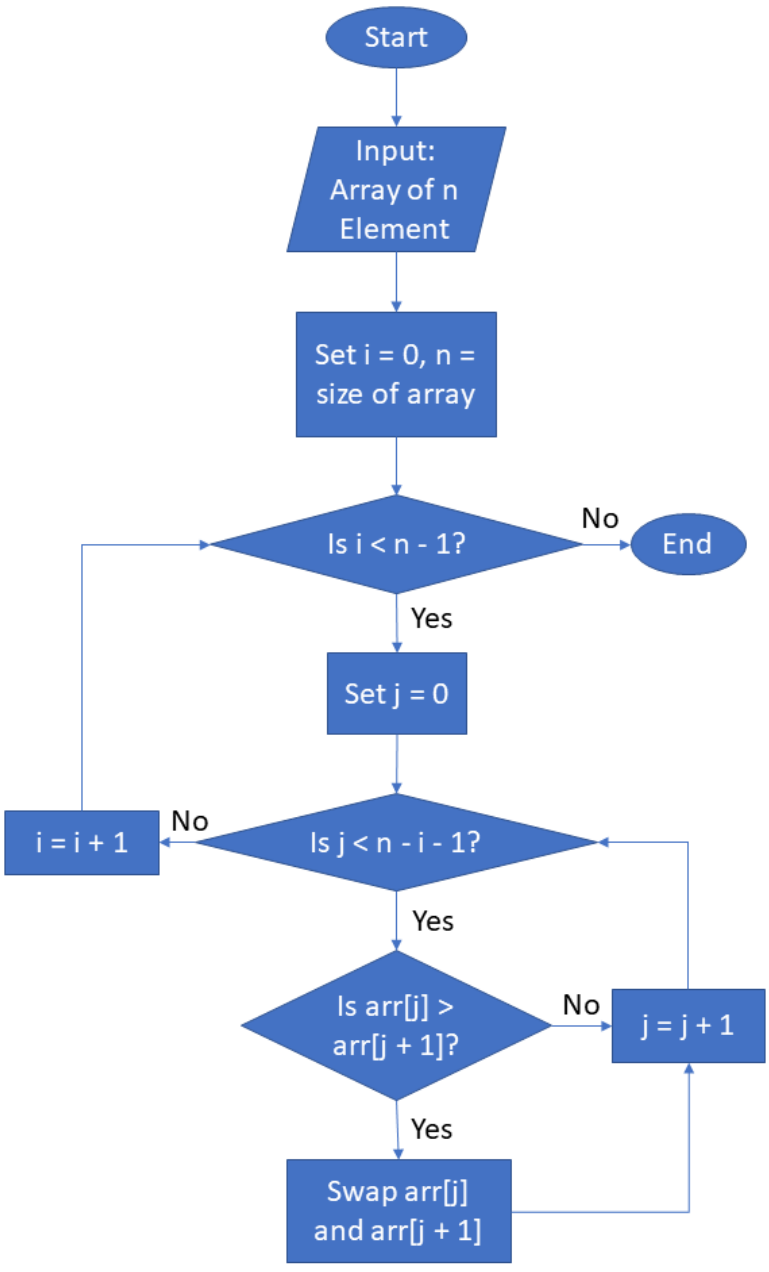
b) Draw a **flowchart** for the **binary search** algorithm.

Ans:



OR
Draw a **flowchart** for the **bubble sort** algorithm.

Ans:



c) What do you mean by **complexity of algorithms**? What is the **time complexity** of the following functions? Explain.

i)

```
void fun1 (int N)
{
    int a = 0, i = N;
    do {
        a += i;
        i /= 2;
    } while (i > 0);
}
```

ii)

```
int fun2 (int n)
{
    int i, j, sum = 0;
    for(i = 0; i * i < n; i++) {
        for (j = 1; j * j <= n; j++) {
            sum += i * j;
        }
    }
    return sum;
}
```

Ans:

i) To determine the time complexity of this code, let's analyze it step by step:

The function “fun1” contains a “do-while” loop:

- The loop starts with i = N and repeatedly divides i by 2 (i /= 2) until i > 0 is no longer true.

- Inside the loop, the operation a += i is performed, which takes constant time, O(1).

Loop Analysis:

- Initially, i = N.

- After the first iteration, i = N / 2.

- After the second iteration, i = N / 4.

- After the third iteration, i = N / 8.

- And so on, until i becomes 0 (since i is an integer, the loop stops when i drops below 1).

The value of i is halved in each iteration, so the number of iterations depends on how many times we can divide N by 2 before reaching a value less than 1. This is a logarithmic process:

- If k is the number of iterations, then after k divisions, i = N / 2^k.

- The loop continues as long as i > 0, i.e., N / 2^k > 0.

- The loop stops when N / 2^k < 1, which means 2^k > N.

- Taking the logarithm base 2, this implies $k > \log_2(N)$.

- Since k is an integer, the loop runs for approximately $\lfloor \log_2(N) \rfloor + 1$ iterations (the +1 accounts for the fact that the loop starts when i = N and includes the case when i becomes 0 after the final division).

Total Time Complexity:

- The loop runs approximately $\log_2(N)$ times (the floor and +1 are typically omitted in big-O notation for simplicity).

- The operation inside the loop (a += i) is O(1).

- Therefore, the total time complexity is **O(log N).**

ii) To determine the time complexity of this code, let's analyze it step by step:

The function fun2 contains two nested loops:

1. The outer loop: for(i = 0; i * i < n; i++)

   - This loop runs as long as i * i < n, which means i goes from 0 up to the largest integer where i < √n.

   - The number of iterations for the outer loop is approximately √n.

2. The inner loop: for (j = 1; j * j <= n; j++)

   - This loop runs as long as j * j <= n, so j goes from 1 up to the largest integer where j ≤ √n.

   - The number of iterations for the inner loop is also approximately √n.

Inside the nested loops, the operation sum += i * j is performed, which takes constant time, O(1).

Total Time Complexity

- For each iteration of the outer loop (which runs √n times), the inner loop also runs √n times.

- Thus, the total number of iterations is √n * √n = n.

- Since the operation inside the loops is O(1), the overall time complexity is **O(n).**

d) Suppose, S1 = “GLADIATOR”, S2 = “DESH”, S3 = “BAN”. Evaluate and write the output of the following operations in order [Assume first Index of the string is 1]:

i) L = LENGTH (S3)
ii) S4 = DELETE (S1, 4, L + 3)
iii) S5 = S3 || S2
iv) S5 = INSERT (S5, 4, S4)
v) INDEX (S5, “NGL”)

Ans: Given:
S1 = "GLADIATOR", S2 = "DESH", S3 = "BAN"
(1-based index assumed)

i)
L = LENGTH(S3) = 3

ii)
S4 = DELETE(S1, 4, L+3) = DELETE(S1, 4, 6)
S1 = "GLADIATOR"
Remove 6 characters starting from 4th index → Remove: "DIATOR"
Remaining: "GLA"

iii)
S5 = S3 || S2 = "BAN" + "DESH" = "BANDESH"

iv)
S5 = INSERT(S5, 4, S4) = INSERT "GLA" at position 4 of "BANDESH"
Result:
Before insert: "BAN|DESH"
After insert: "BANGLADESH"

v)
INDEX(S5, “NGL”) = INDEX(“BANGLADESH”, “NGL”)
Substring “NGL” starts at position 3 (1-based index)

2. a) What is **linear array**? How can we represent a linear array in memory?

Ans:

A linear array is a data structure consisting of a collection of elements arranged in a sequential order, where each element is identified by an index and stored in a consecutive manner, allowing efficient access and manipulation.

In memory, a linear array is represented as a contiguous block of locations. All elements are stored one after another without gaps, starting from a base address, which is the memory location of the first element, typically denoted as `A[0]`. Each element occupies a fixed amount of space determined by its data type—for example, 4 bytes for an integer. The address of any element can be calculated efficiently using the equation: **Address(A[i]) = base + i × size**, where `base` is the memory address of `A[0]`, `i` is the index of the element, and `size` is the size (in bytes) of each element, enabling fast, constant-time access. This contiguous layout is cache-friendly due to spatial locality and supports random access, though it often assumes a fixed size in languages like C, making resizing costly and operations like insertions or deletions in the middle inefficient due to the need to shift elements. This straightforward and efficient memory arrangement is fundamental to how arrays are implemented in programming.

OR
What is **sparse matrix**? Describe how memory can be efficiently utilized by using a sparse matrix representation.

Ans:

A sparse matrix is a matrix where most elements are zero, unlike a dense matrix with mostly non-zero values. It's common in areas like science, graphs, and machine learning, where large matrices have few non-zero entries.

Storing a sparse matrix as a regular 2D array wastes memory because it saves space for all elements, including zeros. Sparse representations save only non-zero elements with their positions, cutting memory use a lot. For example, a 5x5 matrix with 3 non-zero elements takes 100 bytes as a full array (4 bytes per element), but a sparse method needs much less. The Coordinate List (COO) format stores each non-zero element as a triplet—(row, column, value)—so those 3 elements use just 36 bytes (12 bytes per triplet). The Compressed Sparse Row (CSR) format uses three arrays: values, column indices, and row pointers, compressing data even more while keeping access fast. These methods ignore zeros, making memory use depend on non-zero elements (O(k)) instead of total size (O(n×m)), perfect for huge matrices with few non-zeros. The downside is trickier access and operations, but the memory savings and faster zero-skipping math make it worth it for sparse data.

b) Consider a 2D array A (5 : 10, 8 : 15).

i) Find the length of each dimension and the number of elements in A.

ii) Suppose **Base (A) = 200** and **w = 4 words** per memory cell for A. Find the address of the element A[7, 10] in **row-major order**.

Ans: **i)**
Length of rows = 6
Length of columns = 8
Number of elements = 6 × 8 = **48**

ii)
To find address of A[7,10] in **row-major**:

Formula:
Address(A[i][j]) = Base + w * [(i - row_low) * num_cols + (j - col_low)]

Given:
Base = 200, w = 4
row_low = 5, col_low = 8
i = 7, j = 10

Address = 200 + 4 * [(7 - 5) * 8 + (10 - 8)]
= 200 + 4 * [2 * 8 + 2]
= 200 + 4 * (16 + 2)
= 200 + 4 * 18
= 200 + 72
= 272

Answer: 272

c) Let A be an **n x n** square matrix. Write a module which

i) Find the number **NUM of zero** elements in A.

ii) Find the **SUM** of the elements above the diagonal i.e. elements A[I, J] where I < J.

Ans:

i)

```
int countZeros(int A[][N], int n) {
    int count = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (A[i][j] == 0)
                count++;
    return count;
}
```

ii)

```
int sumAboveDiagonal(int A[][N], int n) {
    int sum = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            sum += A[i][j];
    return sum;
}
```

OR

Modify the **bubble sort** algorithm so that it will stop early if it detects that the list is already sorted.

Ans:

```
BUBBLE_SORT (DATA, N)
Here DATA is an array with N elements. This algorithm sorts the elements in DATA.
1. Repeat Steps 2, 3, and 4 for K = 1 to N-1
2. Set PTR := 1
3. Set SWAPPED := False  ⟶ (New step: track if swaps occur)
4. Repeat while PTR <= N-K:
   (a) If DATA[PTR] > DATA[PTR+1], then:
       - Interchange DATA[PTR] and DATA[PTR+1]
       - Set SWAPPED := True
   (b) Set PTR := PTR + 1
5. If SWAPPED = False, Exit  ⟶ (New step: stop early if no swaps)
6. Exit
```

d) Show the steps to search the ITEM = X and ITEM = Y from the following list of integers using *binary search* algorithm.

11, 22, 30, 35, 41, 45, 53, 66, 68, 71, 82, 89, 99

[Here, X is the first two digits and Y is the last two digits of your ID. For example, if ID = C241014, then X = 24 and Y = 16]

Ans:

Binary Search for ITEM = 24:

We apply binary search to search for ITEM = 24.
Initially, **BEG = 1** and **END = 13**

1. **MID = INT((1 + 13)/2) = 7** → DATA[MID] = 53
   Since 24 < 53 → Update: **END = MID - 1 = 6**
2. **MID = INT((1 + 6)/2) = 3** → DATA[MID] = 30
   Since 24 < 30 → Update: **END = MID - 1 = 2**
3. **MID = INT((1 + 2)/2) = 1** → DATA[MID] = 11
   Since 24 > 11 → Update: **BEG = MID + 1 = 2**
4. **MID = INT((2 + 2)/2) = 2** → DATA[MID] = 22
   Since 24 > 22 → Update: **BEG = MID + 1 = 3**

Now, **BEG > END** → Item **24 is not found** in the list.

Binary Search for ITEM = 14:

Again using the same array and starting:
BEG = 1, **END = 13**

1. **MID = INT((1 + 13)/2) = 7** → DATA[MID] = 53
   Since 14 < 53 → Update: **END = MID - 1 = 6**
2. **MID = INT((1 + 6)/2) = 3** → DATA[MID] = 30
   Since 14 < 30 → Update: **END = MID - 1 = 2**
3. **MID = INT((1 + 2)/2) = 1** → DATA[MID] = 11
   Since 14 > 11 → Update: **BEG = MID + 1 = 2**
4. **MID = INT((2 + 2)/2) = 2** → DATA[MID] = 22
   Since 14 < 22 → Update: **END = MID - 1 = 1**

Now, **BEG > END** → **Item 14 is not found in the list.**

Disclaimer: The *Stack* part in Question 3 has been skipped due to syllabus changes — it has now been replaced by *Linked List*.